Implementation and Reduced-Scale Reproduction of *Generative Adversarial Nets*

Surendra Chouhan
Department of Computer Science and Engineering
Rajiv Gandhi Institute of Petroleum Technology, Jais, Amethi, India

***Abstract*—We re-implement the generative adversarial network (GAN) framework of Goodfellow et al. [1] from scratch in PyTorch and test it at reduced scale. Our implementation covers the minimax and non-saturating generator objectives, an MLP generator, a maxout discriminator with dropout, the Parzen-window log-likelihood estimator used in the paper's Table 1, and a controlled 2-D mixture-of-Gaussians testbed in which mode coverage can be measured exactly. On the toy testbed the implementation behaves as the paper describes: training the generator five times per discriminator step reproduces the "Helvetica scenario" (0/8 modes covered), while the minimax and non-saturating losses are statistically indistinguishable over three seeds. On MNIST, however, our reduced-scale reproduction of Table 1 *failed*: the generator collapsed to a single output and obtained a Parzen log-likelihood of -321 against the paper's $+225$. Diagnostics show the discriminator overpowering the generator ($D(G(z)) \to 0$). A single-seed 8k-iteration ablation suggests that a smaller noise dimension or a narrower generator avoids the collapse, but this has not yet been confirmed with multiple seeds. We report all numbers exactly as measured and discuss limitations.**

***Index Terms*—Generative adversarial networks, paper reproduction, PyTorch, mode collapse, ablation study, Parzen-window evaluation**

I. Introduction

Deep generative models aim to learn a distribution p_g that matches an unknown data distribution p_{data} and to draw new samples from it. Before 2014, powerful models such as deep belief networks and deep Boltzmann machines needed either intractable partition-function gradients, Markov-chain sampling, or approximate inference during training [1]. Goodfellow et al. proposed instead to train a generator G by playing it against a discriminator D that tries to tell real data from generated samples. Only back-propagation is needed, and sampling is a single forward pass.

This report documents a from-scratch re-implementation of that framework as a course project. Our objectives are to (i) implement the core algorithm ourselves, (ii) reproduce a key result of the paper at reduced scale, and (iii) test what matters through ablation. Our contributions are:

- a compact, config-driven PyTorch implementation of the minimax and non-saturating GAN objectives (Sec. III);
- a toy testbed with exactly known modes that makes mode collapse measurable (Sec. IV);
- an attempted reproduction of the MNIST column of the paper's Table 1, with an honest account of why it failed (Sec. V);
- ablations of the generator loss and the D:G update ratio (Sec. VI).

II. Background and Related Work

Prior generative models. The paper compares against deep belief networks (DBN), stacked contractive auto-encoders (CAE) and deep generative stochastic networks (GSN), all of which need Markov chains or approximate inference [1]. GANs trade the explicit density $p_g(x)$ for cheap sampling and a flexible model class.

Building blocks. The paper's networks use rectified-linear generators and maxout discriminators [3] trained with dropout [4]. We use Adam [5] instead of the paper's stochastic gradient descent with momentum.

Evaluation. Because p_g is not available in closed form, the paper estimates test log-likelihood by fitting a Gaussian Parzen window to generated samples and choosing its bandwidth on a validation set [2]. The authors themselves note that this estimator has high variance and degrades in high dimensions; later work discusses these weaknesses in detail [9]. Sample diversity is also commonly summarized with an Inception-style score [8], which we adapt to MNIST.

III. Methodology

A. The adversarial game

Let $z \sim p_z$ be noise and $G(z)$ a generated sample. The discriminator outputs $D(x) \in (0,1)$, the probability that x is real. The two networks play the minimax game

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]. \quad (1)$$

In words: D is rewarded for scoring real data high and generated data low, while G is rewarded for fooling D.

B. Optimal discriminator and the global optimum

For a fixed G, write $V = \int p_{\text{data}}(x)\log D(x) + p_g(x)\log(1 - D(x))\,dx$. The integrand $a\log y + b\log(1-y)$ is maximized at $y = a/(a+b)$, so

$$D^*_G(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (2)$$

Substituting (2) back gives the generator criterion

$$C(G) = \max_D V(D,G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \,\|\, p_g), \quad (3)$$

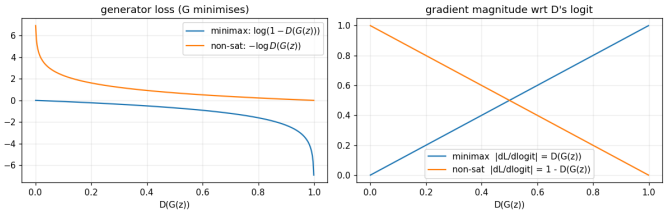


Fig. 1. Generator losses (left) and their gradient magnitude with respect to D's logit (right) as a function of $D(G(z))$. The minimax gradient vanishes exactly where G is weakest.

where JSD is the Jensen–Shannon divergence. It is non-negative and zero only when $p_g = p_{\text{data}}$, so the global optimum is $p_g = p_{\text{data}}$ with $D^* = 1/2$ everywhere [1].

C. Practical losses

Early in training D easily rejects samples, so $D(G(z)) \approx 0$ and $\log(1-D(G(z)))$ is flat, giving G almost no gradient. The paper therefore suggests training G to *maximize* $\log D(G(z))$ instead. With $D(x) = \sigma(a(x))$ for logit a, the losses we implement are (using $\log(1-\sigma(a)) = -\text{softplus}(a)$ and $-\log\sigma(a) = \text{softplus}(-a)$ for numerical stability):

$$\mathcal{L}_D = \mathbb{E}_x[\text{softplus}(-a(x))] + \mathbb{E}_z[\text{softplus}(a(G(z)))], \tag{4}$$

$$\mathcal{L}_G^{\text{minimax}} = -\mathbb{E}_z[\text{softplus}(a(G(z)))], \tag{5}$$

$$\mathcal{L}_G^{\text{non-sat}} = \mathbb{E}_z[\text{softplus}(-a(G(z)))]. \tag{6}$$

The gradient of (5) with respect to the logit has magnitude $D(G(z))$, which vanishes when $D(G(z)) \to 0$, whereas that of (6) has magnitude $1 - D(G(z))$ (Fig. 1).

D. Training procedure

Each iteration performs k discriminator updates on mini-batches of real and generated samples (we use $k = 1$ unless stated), followed by one generator update on fresh noise. D's dropout stays active in both steps. A single training loop, parameterized by a configuration object, is used for every experiment so that ablations change exactly one field.

IV. Experimental Setup

A. Toy 2-D testbed

We use two Gaussian mixtures with component standard deviation $\sigma = 0.05$: `ring8` (8 modes on a circle of radius 2) and `grid25` (5×5 grid with spacing 2). Both networks are 3-layer MLPs with 128 hidden units (G: ReLU; D: LeakyReLU(0.2)), $z \in \mathbb{R}^2$, Adam ($\beta_1 = 0.5$, learning rate 5×10^{-4}), batch size 256, 5000 iterations. On 5000 generated points we report (a) *modes covered*: modes receiving at least 10% of their fair share of high-quality samples; (b) *hq*: the fraction of samples within 4σ of some mode; (c) *JS*: the Jensen–Shannon divergence, normalized to $[0, 1]$, between the distribution of high-quality samples over modes and the uniform distribution.

TABLE I
Toy results, mean ± std over 3 seeds (5000 iterations).

Data	G loss	Modes	hq	JS
ring8	minimax	8.0 ± 0.0	0.949 ± 0.006	0.001 ± 0.000
ring8	non-sat	7.7 ± 0.6	0.960 ± 0.005	0.032 ± 0.049
grid25	minimax	20.3 ± 3.2	0.453 ± 0.102	0.150 ± 0.074
grid25	non-sat	17.7 ± 2.5	0.551 ± 0.132	0.203 ± 0.051

B. MNIST

We use the standard 50k/10k/10k train/validation/test split of MNIST [6] with pixels scaled to $[0, 1]$ and flattened to 784 dimensions. The generator has $z \in \mathbb{R}^{100}$, two ReLU layers of 1200 units and a sigmoid output; the discriminator has two maxout layers (240 units, 5 pieces) with dropout 0.5. Training uses the non-saturating loss, Adam ($\beta_1 = 0.5$, lr 2×10^{-4}), batch size 100 and 20,000 iterations (seed 0). The code is written in PyTorch [7] and was run on a CUDA GPU; each 20k-iteration run took under two minutes.

Parzen-window metric. We fit an isotropic Gaussian kernel density estimate on 10,000 generated samples, choose σ from a 10-point log grid on $[0.1, 1]$ by maximizing mean log-likelihood on the validation set, and report the mean log-likelihood on the test set with its standard error across test examples:

$$\log \hat{p}(x) = \operatorname*{logsumexp}_i \Big(-\tfrac{\|x - g_i\|^2}{2\sigma^2} \Big) - \log N - \tfrac{d}{2}\log(2\pi\sigma^2), \tag{7}$$

with $d = 784$ and g_i the generated samples.

Extra diagnostics. To study collapse we train a small CNN classifier on real MNIST (test accuracy 98.6%) and compute the class histogram of generated digits, the normalized class entropy (1 means all ten classes equally frequent), and an Inception-style score $\exp(\mathbb{E}_x \text{KL}(p(y|x) \| p(y)))$, which is at most 10. On real test data this score is 9.49 and the entropy 0.999.

C. Deviations from the paper

Our setup differs from the paper in: Adam instead of SGD with momentum; shorter training (20k iterations); a single training seed for the main MNIST run; no re-training of the DBN, CAE and GSN baselines (we quote their numbers); no TFD experiment; and the Parzen σ grid of 10 values. Exact numerical agreement is therefore not expected, and the Parzen estimate itself is noisy [9].

V. Results and Discussion

A. Toy results

Table I shows the main comparison over three seeds. On `ring8` both losses cover essentially all modes. On the harder `grid25` neither loss covers all 25 modes, and the difference between losses is smaller than the seed-to-seed spread.

TABLE II
MNIST PARZEN-WINDOW LOG-LIKELIHOOD (HIGHER IS BETTER). TOP: VALUES QUOTED FROM TABLE 1 OF [1]. BOTTOM: OUR MEASUREMENTS (SEED 0, 20K ITERATIONS). IS: INCEPTION-STYLE SCORE (MAX 10); ENT.: NORMALIZED CLASS ENTROPY.

Model	Log-lik.	IS	Ent.
DBN [paper]	138 ± 2	–	–
Stacked CAE [paper]	121 ± 1.6	–	–
Deep GSN [paper]	214 ± 1.1	–	–
Adversarial nets [paper]	225 ± 2	–	–
GAN, non-sat (ours)	-321.4 ± 1.2	1.00	0.018
GAN, minimax (ours)	-435.4 ± 0.5	1.03	0.070
KDE of 10k real images	242.4 ± 2.1	–	–
Untrained generator	-541.3 ± 0.1	1.00	–

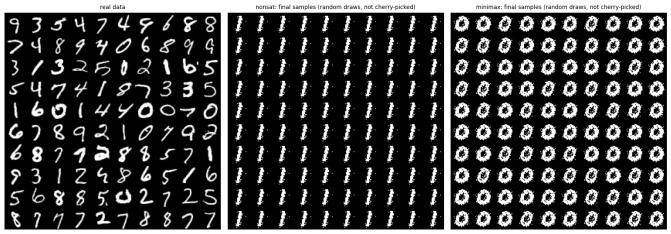


Fig. 2. Real MNIST digits (left) and final samples from our non-saturating (middle) and minimax (right) generators. Both have collapsed. Samples are random draws, not selected.

B. MNIST: comparison with Table 1 of the paper

Table II separates the values quoted from the paper (top) from our own measurements (bottom). Our GAN does not reproduce the paper's result. Both generators collapse: the non-saturating generator outputs one digit "1" for every noise vector and the minimax generator outputs a single blob (Fig. 2); the Inception-style score is ≈ 1 and class entropy is near zero. Two reference rows calibrate our estimator: a KDE built from 10k real training images scores 242.4 ± 2.1, close to the paper's 225, while an untrained generator scores -541.3. The estimator is therefore working; the generator is not.

C. Why the MNIST run failed

The training curves (Fig. 3) show that the discriminator wins decisively: mean $D(G(z))$ falls to nearly zero within the first few thousand iterations and D's loss stays near zero. For the minimax loss, the generator's gradient norm decays by many orders of magnitude, as the saturation argument of Sec. III predicts. The non-saturating loss keeps a large gradient yet still collapses, so avoiding saturation alone is not sufficient for stable training in our setup. We did not find a coding error in the losses, but we cannot rule out that our optimizer and capacity choices make D too strong.

VI. ABLATION STUDY

A. Hypotheses

H1 (loss). The paper argues that the non-saturating loss gives G a useful gradient where the minimax loss saturates; we therefore expected it to cover more modes or score higher.

TABLE III
MNIST ONE-FACTOR ABLATION, 8000 ITERATIONS, *one seed* (PRELIMINARY). BASE: NON-SAT LOSS, z=100, G WIDTH 1200, DROPOUT 0.5, MAXOUT D, D:G = 1:1, ADAM β_1=0.5.

Factor	Setting	Log-lik.	IS	Ent.
–	base	-295.1	1.00	0.004
Loss	minimax	-410.1	1.02	0.051
D dropout	0.0	-445.0	1.01	0.227
D activation	leaky ReLU	-350.1	1.01	0.287
D:G steps	3:1	-546.9	1.00	0.005
D:G steps	1:3	-331.6	1.00	0.113
z dim	10	$\mathbf{103.9}$	5.87	0.894
G width	512	-524.3	1.00	0.059
G width	128	-131.1	1.94	0.466
Adam β_1	0.0	-796.9	1.00	0.032
Adam β_1	0.9	-682.2	1.00	0.597

H2 (update ratio). The paper warns that G must not be trained too much without updating D (the "Helvetica scenario"); we expected a G-heavy ratio to collapse.

B. Toy results

For H1, Table I does *not* support an advantage for the non-saturating loss on the toy data: on `grid25` minimax covers slightly more modes (20.3 ± 3.2 vs. 17.7 ± 2.5), a difference within one standard deviation. A plausible reason, consistent with our diagnostics, is that mean $D(G(z))$ stayed roughly between 0.2 and 0.5 throughout toy training (Fig. 4), so the saturating regime was never reached. With only three seeds we treat this as inconclusive rather than as evidence against the paper.

For H2, on `ring8` (seed 0, non-saturating loss), a 1:5 ratio ($k = 1$ discriminator step, 5 generator steps) collapsed to a single point that hops between modes (0/8 modes, JS $= 1.0$), while the balanced 1:1 run covered 8/8 modes (Fig. 5). This matches the paper's description. A 5:1 discriminator-heavy ratio gave 2/8 modes by our strict metric (hq 0.93, JS 0.50); the samples trace the circle but fail to settle on modes. In the larger `grid25` ablation (2 seeds) the 1:5 ratio also collapsed (0.5 and 1.5 modes for minimax and non-saturating, respectively).

C. MNIST ablation (preliminary)

Table III varies one factor at a time from the base configuration, trained for 8000 iterations with a *single seed*. Because the base configuration itself is collapsed (score 1.00), these numbers should be read only as leads. Two changes stand out: reducing the noise dimension from 100 to 10 gave a Parzen log-likelihood of $+103.9$ with score 5.87 and entropy 0.89, and a narrower generator (128 units) gave -131.1 with score 1.94. Every other change left the score near 1. Without repeated seeds we cannot say whether these gains are robust.

VII. LIMITATIONS AND FAILURE CASES

- **Failed reproduction.** Our MNIST GAN collapsed (Sec. V); the -321 vs. $+225$ comparison is not a valid reproduction of the paper's claim, only a record of a failed attempt.

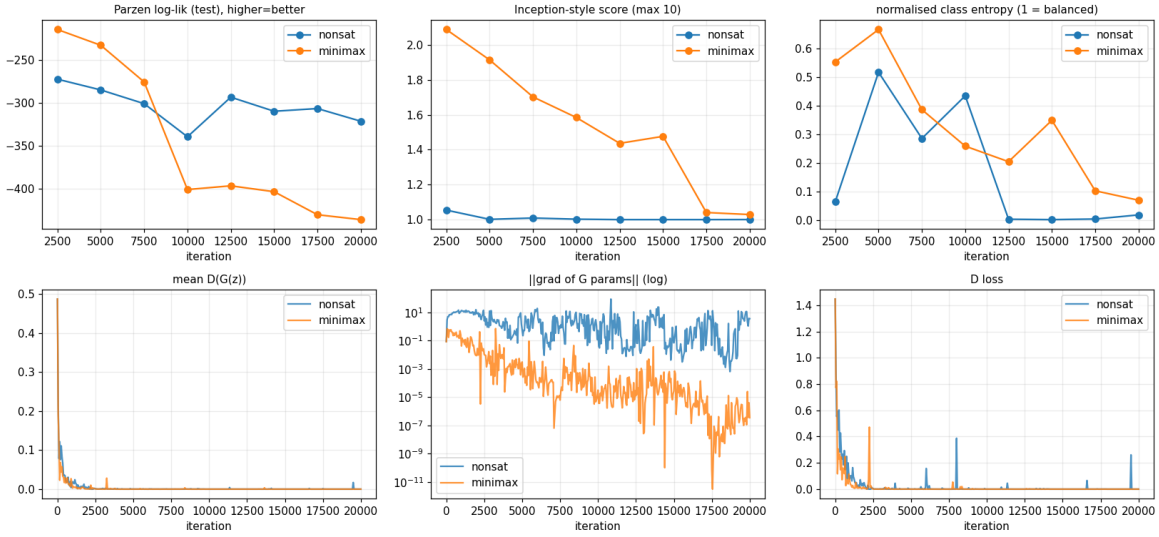


Fig. 3. MNIST training diagnostics for both losses (top: Parzen log-likelihood, Inception-style score, class entropy at eight checkpoints; bottom: $D(G(z))$, generator gradient norm, discriminator loss).

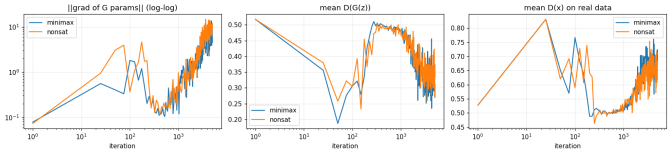


Fig. 4. Toy (`grid25`) diagnostics: generator gradient norm, mean $D(G(z))$ and mean $D(x)$. Mean $D(G(z))$ stays well above zero.

- **Few seeds.** The main MNIST run and the whole MNIST ablation use one seed; the toy ablation uses two or three. GANs are noisy, so small differences are not reliable.
- **Optimizer and training length.** We use Adam and 20k iterations rather than the paper's SGD with momentum, and we did not tune the base MNIST configuration on the validation set before running ablations.
- **Metric weakness.** The Parzen estimator has high variance and is unreliable in 784 dimensions [1], [9]. The $+103.9$ result in Table III should not be compared to 225 without repeated runs.
- **Scope.** We did not retrain the DBN, CAE or GSN baselines and did not attempt the TFD or CIFAR-10 experiments. Our mode-coverage metric on the toy data depends on the 4σ and 10% thresholds we chose.

VIII. Conclusion and Future Work

We implemented the GAN objective, a maxout discriminator and the Parzen evaluation pipeline from scratch and verified the paper's qualitative warning about unbalanced updates on a toy problem. We could not reproduce the paper's MNIST Parzen result: both of our generators collapsed, with the discriminator overpowering the generator. A reduced noise dimension or generator width looks like a promising fix, but this is a single-seed observation. Future work is to select the base configuration using only validation data, run at least three seeds, and repeat the loss ablation on a healthy MNIST baseline before drawing conclusions about H1.

Reproducibility Note

The numerical results in this report are reported as measured in the project notes supplied by the author. They should be interpreted as a reduced-scale experimental attempt, not as an independently verified reproduction of the original paper. Archive the source code, raw logs, and figure assets with the final submission so that the reported results can be independently checked.

Code and Attribution

All model, loss and training code was written by the author for this project using standard PyTorch layers, optimizers and the torchvision MNIST loader. No third-party GAN implementation is reproduced here.

References

[1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems*, vol. 27, 2014, arXiv:1406.2661.

[2] O. Breuleux, Y. Bengio, and P. Vincent, "Quickly Generating Representative Samples from an RBM-Derived Process," *Neural Computation*, vol. 23, no. 8, pp. 2058–2073, 2011.

[3] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout Networks," in *Proceedings of the International Conference on Machine Learning*, 2013, arXiv:1302.4389.

[4] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. S. Salakhutdinov, "Improving Neural Networks by Preventing Co-adaptation of Feature Detectors," arXiv:1207.0580, 2012.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the International Conference on Learning Representations*, 2015, arXiv:1412.6980.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[7] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

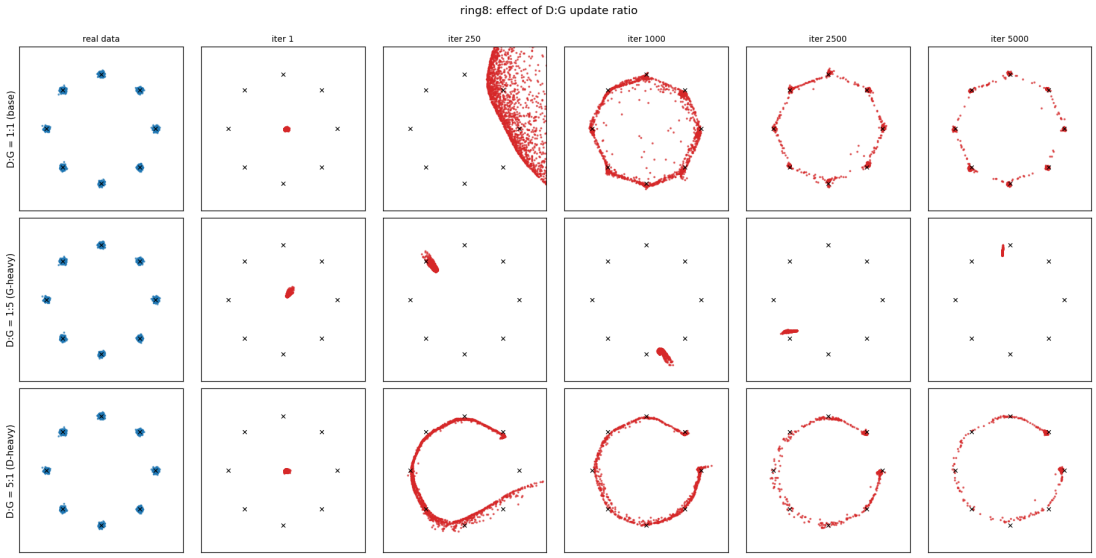


Fig. 5. Effect of the D:G update ratio on `ring8` (seed 0, non-saturating loss). Red: generated samples; crosses: true modes. The G-heavy run collapses to a point, as in the paper's "Helvetica scenario".

[8] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved Techniques for Training GANs," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[9] L. Theis, A. van den Oord, and M. Bethge, "A Note on the Evaluation of Generative Models," in *Proceedings of the International Conference on Learning Representations*, 2016.